



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

补充讲义：抽象工厂在嵌入式驱动层的局限性

1、之前讲义的例子回顾

- 描述了抽象工厂用于 MCU 平台选择（STM32/GD32/NXP）
- 假设业务代码只依赖抽象工厂创建的 UART/GPIO/SPI 对象
- Client 完全不感知具体平台，实现平台无关业务逻辑

理论上，这符合 GoF 抽象工厂定义：一次选择，创建一组成套对象族。

2、实际嵌入式工程问题

2.1 驱动层不需要抽象工厂

假设	实际情况
每个外设（UART/GPIO/SPI）由工厂创建	MCU HAL 或 RTOS 设备框架本身提供统一接口 + 函数表 + 注册机制
业务逻辑不关心具体平台	HAL/RTOS 驱动接口已解耦寄存器和具体芯片型号，业务直接调用即可
平台切换通过 ConcreteFactory	实际 MCU 工程通常通过编译选项、BSP 选择或宏定义实现；运行期动态切换不常见

结论：对底层驱动和外设对象使用抽象工厂会导致**结构复杂、额外抽象层、运行期开销增加**，而工程上没有实质收益。

2.2 抽象工厂与 MCU HAL / RTOS 设备框架对比

特性	抽象工厂设计	HAL / RTOS / Linux 驱动模型
对象创建	ConcreteFactory 负责实例化成套 ConcreteProduct	驱动注册/设备表 + ops 表，业务直接调用接口
平台切换	初始化期选择 ConcreteFactory	编译期 BSP 或 Makefile 选择，运行期一般固定
可维护性	增加工厂/抽象产品数量，代码量大	驱动接口统一，易扩展且贴合 MCU 架构
灵活性	可动态替换对象族，但嵌入式不常用	支持动态绑定驱动和设备（Linux/RTOS）
适用场景	多 SKU / 多平台产品族，方案初始化期绑定	单平台 MCU 或 HAL 层抽象，外设调用一致接口即可

3、现实做法建议

- 1. 驱动层/底层外设
 - 统一接口 + 函数表 + 注册机制
 - MCU HAL / LL / RTOS device model
 - 避免为每个 UART/GPIO 创建工厂
- 2. 算法/策略可替换
 - 使用 Strategy 模式
 - 适合运行期选择控制算法或业务策略
- 3. 产品线/方案族切换
 - 适合抽象工厂
 - 初始化期选择 ConcreteFactory，创建一整套配套模块
 - 典型例子：智能家居不同 SKU（Basic/Pro/Industrial）组合显示、通信、执行器

4、总结

- 之前讲义例子的问题：把抽象工厂用于每个 MCU 外设创建，实际嵌入式工程不需要，也不符合 HAL/RTOS 实际模式。
- 核心理念依然有价值：保证“同一方案/平台下对象族一致性”，适合产品族层面。
- 底层驱动抽象不适合工厂：统一接口 + 注册/ops 表更贴近工程实践。